

Context Switch

- When an interrupt occurs, the system needs to save the current context of the process running on the CPU so that it can restore that context when its processing is done.
- Switching the CPU to another process requires performing a state save of the current process and a state restore of a different process. This task is known as a context switch. When a context switch occurs, the kernel saves the context of the old process in its PCB and loads the saved context of the new process scheduled to run. Context-switch time is pure overhead, because the system does no useful work while switching.
- Switching speed varies from machine to machine, depending on the memory speed, the number of registers that must be copied, and the existence of special instructions (such as a single instruction to load or store all registers). A typical speed is a few milliseconds.

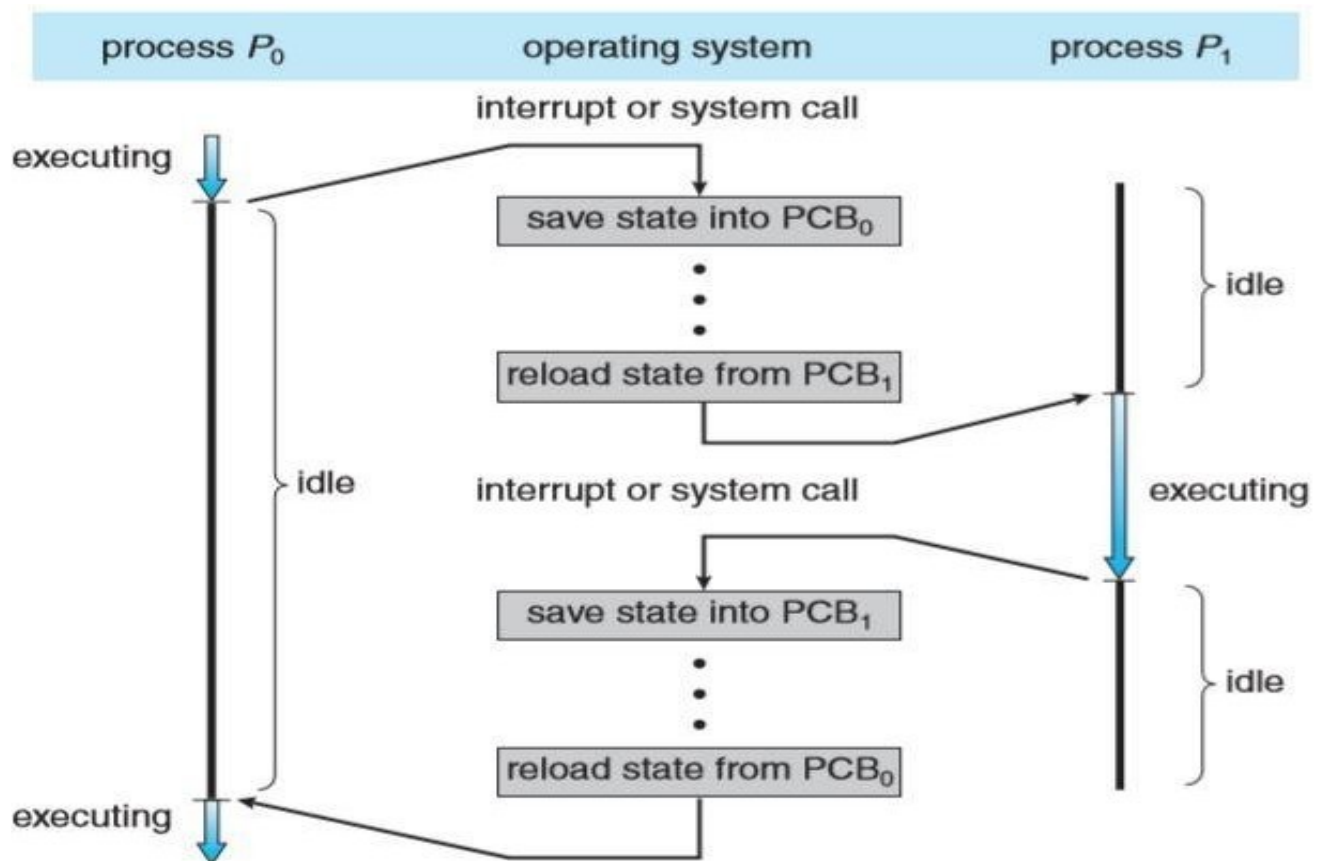


Diagram showing CPU switch from process to process.

CPU Scheduling

- A process execution consists of a cycle of CPU execution or wait and i/o execution or wait. Normally a process alternates between two states. Process execution begin with the CPU burst that may be followed by a i/o burst, then another CPU and i/o burst and so on. eventually in the last will end up on CPU burst. So, process keep switching between the CPU and i/o during execution.

Terminology

- o **Arrival Time (AT)**: Time at which process enters a ready state.
- o **Burst Time (BT)**: Amount of CPU time required by the process to finish its execution.
- o **Completion Time (CT)**: Time at which process finishes its execution.
- o **Turn Around Time (TAT)**: Completion Time (CT) - Arrival Time (AT), $WT + BT$
- o **Waiting Time**: Turn Around Time (TAT) - Burst Time (BT)
- o **Response Time**: Is the time it takes to start responding, not the time it takes to output the response.

Scheduling criteria

Different CPU-scheduling algorithms have different properties, and the choice of a particular algorithm may favour one class of processes over another. So, in order to efficiently select the scheduling algorithms following criteria should be taken into consideration:

- **CPU utilization**: Keeping the CPU as busy as possible.
- **Throughput**: If the CPU is busy executing processes, then work is being done. One measure of work is the number of processes that are completed per time unit, called throughput.
- **Turnaround time**: The interval from the time of submission of a process to the time of completion is the turnaround time.
- **Waiting time**: Waiting time is the sum of the periods spent waiting in the ready queue.
- **Response Time**: Is the time it takes to start responding, not the time it takes to output the response.
- Note: The CPU-scheduling algorithm does not affect the amount of time during which a process executes I/O; it affects only the amount of time that a process spends waiting in the ready queue. It is desirable to maximize CPU utilization and throughput and to minimize turnaround time, waiting time, and response time.

Type of scheduling

- CPU scheduling decision may take place under the following circumstances:
- **Non-Pre-emptive:** Under Non-Pre-emptive scheduling, once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or by switching to the waiting state. i.e. A process will leave the CPU willingly it can't be forced out. They are rigid as even if a critical process enters the ready queue the process running CPU is not disturbed.
 - When a process completes its execution
 - When a process leaves CPU voluntarily to perform some i/o or other operations and enters waiting state.
- **Pre-emptive**
 - If a new process enters in the ready state, in case of high priority
 - When process switches from running to ready state because of time quantum expire.

FCFS (FISRT COME FIRST SERVE)

- FCFS is the simplest scheduling algorithm, as the name suggest, the process that requests the CPU first is allocated the CPU first. Implementation is managed by FIFO Queue. It is always non pre-emptive in nature.

Example: Consider the following table of arrival time and burst time for three processes P₀, P₁, P₂, P₃, P₄. What is the average Waiting Time and Turnaround Time for the five processes?

P. No	Arrival Time (AT)	Burst Time (BT)
P ₀	0	4
P ₁	1	3
P ₂	2	1
P ₃	3	2
P ₄	4	5

Ans. We make a Gantt chart (Bar chart that illustrates a particular Schedule).

P ₁ (0-4)	P ₂ (4-7)	P ₃ (7-8)	P ₄ (8-10)	P ₅ (10-15)
----------------------	----------------------	----------------------	-----------------------	------------------------

Calculating the various parameters:

P. No	AT	BT	CT	TAT	WT)
P ₀	0	4	4	4	0
P ₁	1	3	7	6	3
P ₂	2	1	8	6	5
P ₃	3	2	10	7	5
P ₄	4	5	15	11	6
Total				34	19

Average TAT = $34/5 = 6.8$

Average WT = $19/5 = 3.8$

Example: Calculating the various parameters:

P. No	AT	BT	CT	TAT	WT
P ₀	0	3			
P ₁	2	2			
P ₂	6	4			
Average					

4

Example: Calculating the various parameters:

P. No	AT	BT	CT	TAT	WT
P ₀	2	4			
P ₁	1	2			
P ₂	0	3			
P ₃	4	2			
P ₄	3	1			
Average					

Example: Calculating the various parameters:

P. No	AT	BT	CT	TAT	WT
P ₀	6	4			
P ₁	2	5			
P ₂	3	3			
P ₃	1	1			
P ₄	4	2			
P ₅	5	6			
Average					

- **Advantage**
 - Easy to understand, and can easily be implemented using Queue data structure.
 - Can be used for Background processes where execution is not urgent.
- **Disadvantage**
 - FCFS suffers from convoy which means smaller process have to wait larger process, which result into large average waiting time. This effect results in lower CPU and device utilization than might be possible if the shorter processes were allowed to go first.
 - This algo fails with time sharing system or interactive systems
 - Higher average waiting time and TAT compared to other algorithms.
 - The FCFS algorithm is thus particularly troublesome for time-sharing systems (due to its non-pre-emptive nature), where it is important that each user get a share of the CPU at regular intervals.

Shortest Job First (SJF)(non-pre-emptive) Shortest Remaining Time First (SRTF)/ (Pre-emptive)

- Whenever we make a decision of selecting the next process for CPU execution, out of all available process, CPU is assigned to the process having smallest burst time requirement. When the CPU is available, it is assigned to the process that has the smallest next CPU burst. If there is a tie, FCFS is used to break tie
- It supports both version non-pre-emptive and pre-emptive (purely greedy approach)
- In Shortest Job First (SJF)(non-pre-emptive) once a decision is made and among the available process, the process with the smallest CPU burst is scheduled on the CPU, it cannot be pre-empted even if a new process with the smaller CPU burst requirement then the remaining CPU burst of the running process enter in the system
- In Shortest Remaining Time First (SRTF) (Pre-emptive) whenever a process enters in ready state, again we make a scheduling decision whether, this new process with the smaller CPU burst requirement then the remaining CPU burst of the running process and if it is the case then the running process is pre-empted and new process is scheduled on the CPU.
- This version (SRTF) is also called optimal as it guarantee minimal average waiting time.

Example on SJF (Non-Pre-emptive), Calculate Average TAT and WT?

P. No	A.T	B.T
P ₁	1	7
P ₂	2	5
P ₃	3	1
P ₄	4	2
P ₅	5	8

Ans. Constructing the Gantt chart

P ₀ (0-1)	P ₁ (1-8)	P ₃ (8-9)	P ₄ (9-11)	P ₂ (11-16)	P ₅ (16-24)
-------------------------	-------------------------	-------------------------	--------------------------	---------------------------	---------------------------

P. No	A.T	B.T	C.T	TAT	WT
P ₁	1	7	8	7	0
P ₂	2	5	16	14	9
P ₃	3	1	9	6	5

P4	4	2	11	7	5
P5	5	8	24	19	11
Total				53	30

Average Waiting Time: $30/5 = 6$

Average Turn Around Time: $53/5 = 10.6$

Example: Consider the following processes:

P. No	AT	BT
P1	0	10
P2	3	6
P3	7	1
P4	8	3

Calculate Average WT and TAT?

Ans.

P1 (0-3)	P2 (3-7)	P3 (7-8)	P2 (8-10)	P4 (10-13)	P1 (13-20)
--------------------	--------------------	--------------------	---------------------	----------------------	----------------------

P. No	AT	BT	CT	TAT	WT
P1	0	10	20	20	10
P2	3	6	10	7	1
P3	7	1	8	1	0
P4	8	3	13	5	2
Total				33	13

Average TAT: $33/4 = 8.25$

Average WT: $13/4 = 3.25$

Example:

Process	AT	BT	CT	TAT	WT
P₁	0	6			
P₂	1	4			
P₃	2	3			
P₄	3	1			
P₅	4	2			

P₆	5	1			
----------------------	----------	----------	--	--	--

Example:

Process	AT	BT	CT	TAT	WT
P ₁	3	4			
P ₂	4	2			
P ₃	5	1			
P ₄	2	6			
P ₅	1	8			
P ₆	2	4			

Advantage:

- 0 Pre-emptive version guarantees minimal average waiting time so some time also referred as optimal algorithm.
- 0 Provide a standard for other algo in terms of average waiting time
- 0 Provide better average response time compare to FCFs

Disadvantage

- 0 This algo cannot be implemented as there is no way to know the length of the next CPU burst.
- 0 Here process with the longer CPU burst requirement goes into starvation.
- 0 No idea of priority, longer process has poor response time.
- **Note:** - There are several approaches either to make SJF implementable or to improve it in terms of starvation or in other sense. Will discuss them later.
- As SJF is not implementable, we can use the one technique where we try to predict the CPU burst of the next coming process. The method is used as exponential averaging technique, where we consider the previous value and previous prediction.
- $\tau_{(n+1)} = \alpha t_n + (1 - \alpha) \tau_n$. This idea is also more of theoretical importance as most of the time the burst requirement of the coming process may vary by a large extent and if the burst time requirement of all the process is approximately same then there is no advantage of using this scheme.

- **Example:**

Process	t	tau
P ₁	10	20
P ₂	12	
P ₃	14	
P ₄		

Priority Scheduling

- Here a priority is associated with each process. At any instance of time out of all available process, CPU is allocated to the process which possess highest priority (may be higher or lower number). Tie is broken using FCFS order. No importance to senior or burst time. It supports both non-pre-emptive and pre-emptive versions.
- In Priority (non-pre-emptive) once a decision is made and among the available process, the process with the highest priority is scheduled on the CPU, it cannot be pre-empted even if a new process with higher priority more than the priority of the running process enter in the system.
- In Priority (pre-emptive) once a decision is made and among the available process, the process with the highest priority is scheduled on the CPU, and if it a new process with higher priority more than the priority of the running process enter in the system, then we do a context switch and the processor is provided to the new process with higher priority.
- Priorities are generally indicated by some fixed range of numbers, such as 0 to 7 or 0 to 4,095. There is no general agreement on whether 0 is the highest or lowest priority, it can vary from systems to systems.

Example: We will assume in example that low numbers have higher priority. Non-Pre-emptive Priority Scheduling

P. No	AT	BT	Priority
P ₁	0	10	3
P ₂	0	1	1
P ₃	0	2	4
P ₄	0	1	5
P ₅	0	5	2

Gantt chart:

P ₂ (0-1)	P ₅ (1-6)	P ₁ (6-16)	P ₃ (16-18)	P ₄ (18-19)
-------------------------	-------------------------	--------------------------	---------------------------	---------------------------

P. No	AT	BT	Priority	CT	TAT	WT
P ₁	0	10	3	16	16	6
P ₂	0	1	1	1	1	0
P ₃	0	2	4	18	18	16
P ₄	0	1	5	19	19	18
P ₅	0	5	2	6	6	1
Total					60	41

Average WT: $41/5 = 8.2$ ms

Average TAT: $60/5 = 12$ ms

Q Example: Pre-emptive priority scheduling

P. No	AT	BT	Priority
P ₁	1	4	4
P ₂	2	2	5
P ₃	2	3	7
P ₄	3	5	8(H)
P ₅	3	1	5
P ₆	4	2	6

Ans. Gantt chart

P ₀ (0-1)	P ₁ (1-2)	P ₃ (2-3)	P ₄ (3-4)	P ₄ (4-8)	P ₃ (8-10)	P ₆ (10-12)	P ₂ (12-14)	P ₅ (14-15)	P ₁ (15-18)
-------------------------	-------------------------	-------------------------	-------------------------	-------------------------	--------------------------	---------------------------	---------------------------	---------------------------	---------------------------

P. No	AT	BT	Priority	CT	TAT	WT
P ₁	1	4	4	18	17	13
P ₂	2	2	5	14	12	10
P ₃	2	3	7	10	8	5
P ₄	3	5	8	8	5	0
P ₅	3	1	5	15	12	11
P ₆	4	2	6	12	8	6
Total					62	45

Average TAT: $62/6 = 10.33$

Average WT: $45/6 = 7.5$

Q Example: Pre-emptive priority scheduling

Process	AT	BT	Priority	CT	TAT	WT
P ₁	0	50	4			
P ₂	20	20	1(H)			
P ₃	40	100	3			
P ₄	60	40	2			

Q Example: Pre-emptive priority scheduling

Process	AT	BT	Priority	CT	TAT	WT
P ₁	1	4	5			
P ₂	2	5	2			

P₃	3	6	6			
P₄	0	1	4			
P₅	4	2	1			
P₆	5	3	8(H)			

- **Advantage**
 - Gives a facility specially to system process.
 - Allow us to run important process even if it is a user process.
- **Disadvantage**
 - Here process with the smaller priority may starve for the CPU
 - No idea of response time or waiting time.
- Note: - Specially use to support system process or important user process
- Ageing: - a technique of gradually increasing the priority of processes that wait in the system for long time. E.g. priority will increase after every 10 mins

Round robin

- This algo is designed for time sharing systems, where it is not, necessary to complete one process and then start another, but to be responsive and divide time of CPU among the process in the ready state. Here ready queue is treated as a circular queue (FIFO). It is similar to FCFS scheduling, but pre-emption is added to enable the system to switch between processes.
- The ready queue is treated as a circular queue. The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval equivalent 1 Time quantum (where value of TQ can be anything).
- We fix a time quantum, up to which a process can hold the CPU in one go, with in which either a process terminates or process must release the CPU and enter the ready queue and wait for the next chance.
- The process may have a CPU burst of less than given time quantum. In this case, the process itself will release the CPU voluntarily. CPU Scheduler will select the next process for execution. OR The CPU burst of the currently running process is longer than 1-time quantum, the timer will go off and will cause an interrupt to the operating system. A context switch will be executed, and the process will be put at the tail of the ready queue. The CPU scheduler will then select the next process in the ready queue.
- RR is always pre-emptive in nature.
- If there are n processes in the ready queue and the time quantum is q, then each process gets $1/n$ of the CPU time in chunks of at most q time units. Each process must wait no longer than $(n - 1) \times q$ time units until its next time quantum.
- If the time quantum is extremely large, the RR policy is the same as the FCFS policy. If the time quantum is extremely small (say, 1 millisecond), the RR approach is called processor sharing and (in theory) creates the appearance that each of n processes has its own processor running at $1/n$ the speed of the real processor. We also need also to consider the effect of context switching on the performance of RR scheduling.
- We have only one process of 10-time units. If the quantum is 12-time units, the process finishes in. less than 1-time quantum, with no overhead. If the quantum is 6-time units, however, the process requires 2 quanta, resulting in a context switch. If the time quantum is 1-time unit, then nine context switches will occur, slowing the execution of the process accordingly. In General, If TQ is less the number of Context Switch increases, response time will be less. If TQ is large the number of Context Switch will decrease and response time increases.
- Optimal Time Quantum, so that every problem gets CPU after t seconds: $q \leq t - ns / n-1$. A rule of thumb is that 80 percent of the CPU bursts should be shorter than the time quantum.

Q Example: Let Time Quantum (TQ) = 2

P. No

AT

BT

P₁	0	4
P₂	1	5
P₃	2	2
P₄	3	1
P₅	4	6
P₆	6	3

Gantt Chart:

P1 (0-2)	P2 (2-4)	P3 (4-6)	P1 (6-8)	P4 (8-9)	P5 (9-11)	P2 (11-13)	P6 (13-15)	P5 (15-17)	P2 (17-18)	P6 (18-19)	P5 (19-21)
--------------------	--------------------	--------------------	--------------------	--------------------	---------------------	----------------------	----------------------	----------------------	----------------------	----------------------	----------------------

P. No	AT	BT	CT	TAT	WT
P₁	0	4	8	8	4
P₂	1	5	18	17	12
P₃	2	2	6	4	2
P₄	3	1	9	6	5
P₅	4	6	21	17	11
P₆	6	3	19	13	10
Total				65	44

Average TAT: $65/6 = 10.83$ ms

Average WT = $44/6 = 7.33$ ms

Process	Arrival Time	Burst Time	CT	TAT	WT
P₁	5	5			
P₂	4	6			
P₃	3	7			
P₄	1	9			
P₅	2	2			
P₆	6	3			

• **Advantage**

- Perform best in terms of average response time
- Works well in case of time-sharing systems, client server architecture and interactive system

- kind of SJF implementation
- **Disadvantage**
 - Longer process may starve
 - Performance depends heavily on time quantum - If value of the time quantum is very less, then it will give lesser average response time (good but total no of context switches will be more, so CPU utilization will be less), If time quantum is very large then average response time will be more bad, but no of context switches will be less, so CPU utilization will be good.
 - No idea of priority